A Mini-Project Report
On

Speech Therapy Web App

By

Name of the student:	Roll number
Samarth Mhatre	125B1B351
Rishabh Jain	125B1B354
Aatmaja Agrawal	125B1B349
Snehal Wavare	125B1B346

Under The Guidance Of

Dr. Mahalalkhsmi Bodireddy



Department of AS & H
PIMPRI CHINCHWAD EDUCATION TRUST's
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING,
Nigdi,Pune- 411044

[An Autonomous institute affiliated to Savitribai Phule Pune University]

Table of Contents

Sr. No	Topic
1.	Introduction
2.	Problem Statement
3.	Objectives of the Project
4.	Software/Hardware Requirements
5.	Algorithm
6.	Flowchart
7.	Program Code
9.	Input/Output
10.	Result and Observation
11.	Learning Outcome
12.	Conclusion
13.	References

Evaluation Sheet

Department of Computer Engineering						
Class : FY - L	Semester: I					Year: 2025-26
Name of the Course	CPPS/DECO		Name of the Faculty:			
Mini Project		1	2	3	4	Total Marks
PRN	Name of the Student	5M	5M	5M	5M	20M
125B1B349	Aatmaja Agrawal	5	3	4	5	17
125B1B346	Snehal Wavare	5	3	4	5	17
125B1B351	Samarth Mhatre	5	3	4	5	17
125B1B354	Rishabh Jain	5	3	4	5	17

5/1/2026

B. Mahalakshmi

Project Guide Signature

Report on prototype for a speech therapy app

1. Introduction

1.1.What is the project about?

The project's aim is to develop a web-application that aids in speech therapy as a supplement alongside professional help. It prompts the user with words and pronunciations based on the type of lisp they select, and takes their pronunciation of it as input to analyze for accuracy.

1.2. Why was this project chosen?

Speech therapy helps people of all ages to improve speech clarity, cognitive skills, and overall communication. It's helpful for people with speech disorders, language disorders, cognitive-communicative issues, as well as people who face difficulty swallowing safely, whether it be due to developmental delays or injuries. Speech therapy helps patients go through life with more confidence and helps with overall well-being.

2.Problem statement

- Speech therapy services are not easily accessible to everyone due to cost and availability issues. There is a need for a simple and user-friendly mobile application that helps users practice speech exercises and improve pronunciation through guided activities and basic feedback.
- The problem is to design and develop a Speech Therapy App prototype that provides basic speech practice exercises, audio guidance, and simple feedback to help users improve their pronunciation and fluency. The app should be easy to use, affordable, and

accessible anytime, allowing users to practice speech exercises independently or alongside professional therapy.

Inputs and outputs

Inputs like Voice input through the device microphone while pronouncing words or sentences. User selection input such as Selecting letters, words, or exercises. Choosing difficulty level. Button inputs like start, stop, repeat, and submit. Optional text input for selecting or typing practice words.

Outputs like audio output of correct pronunciation for selected words or sentences.Visual feedback showing correct or incorrect pronunciation. Score or accuracy percentage (basic level). Text feedback such as Good pronunciation,Try again.Progress display showing completed exercises and improvement level.Encouraging messages to motivate the user.

Real-Life Relevance

- Reduces dependency on frequent visits to speech therapists.
- Provides affordable and accessible speech practice at home.
- Helpful for parents to support their child's speech development.
- Helps children and adults improve speech and pronunciation in daily life.
- Useful for people with speech disorders who need regular practice.
- be used in schools and learning centers for pronunciation training.
- Beneficial in rural or remote areas with limited therapy facilities.
- Improves communication skills and confidence in real-life situations.

---

3. Objectives of the Project

- To understand the basic concepts of speech therapy.
- To design a simple and user-friendly speech therapy application prototype.

- To provide pronunciation practice through guided exercises.
- To accept voice input and give basic feedback to the user.
- To help users improve speech clarity and confidence.

---

4. Software / Hardware Requirements

- The system is a web-based application, so it only needs a good internet connection and a modern web browser.
- Browsers like Google Chrome, Microsoft Edge, Brave, or Mozilla Firefox can be used for development and testing.
- The frontend of the website is created using HTML, CSS, and JavaScript.
- HTML is used to build the structure of the website.
- CSS is used to design and style the web pages.
- JavaScript is used to add interactivity and handle user actions.
- Python is used on the backend to manage basic server operations and act as a mock database.
- The system uses OpenAI's Whisper API for speech processing.
- A microphone is needed to record the user's voice for speech practice.
- Headphones or speakers are used to listen to instructions and correct pronunciation.
- The website works on laptops, desktops, and mobile phones.
- Users can access the website easily through a web browser.

5. Algorithm

I. Architecture Overview

- **Frontend:** HTML/JavaScript pages for user interaction (signup.html, login.html, selection.html, s.html).
- **Backend:** A Python Flask server (app.py) that handles authentication and processes audio.
- **AI Service:** OpenAI Whisper API (used for converting speech to text).
- **Database:** An in-memory list (users_db) acting as a temporary mock database.

II. User Authentication Flow

This phase handles user onboarding and access control.

1. Sign Up Algorithm (signup.html → app.py)

1. **Input:** User enters Full Name, Email, Username, and Password.
2. **Client Validation:** JavaScript checks if "Password" and "Confirm Password" match.
3. **Request:** Frontend sends a POST request to /signup with the user data as JSON.
4. **Server Logic:**
    - Checks if all fields are present.
    - Iterates through users_db to ensure the email is not already registered.
    - **If Valid:** Creates a new user dictionary with an ID and appends it to users_db. Returns HTTP 201 (Created).
    - **If Invalid:** Returns HTTP 409 (Conflict) or 400 (Bad Request).
5. **Output:** Frontend displays a success message.

2. Login Algorithm (login.html → app.py)

1. **Input:** User enters Email and Password.
2. **Request:** Frontend sends a POST request to /login.
3. **Server Logic:**
    - Searches users_db for a user matching the provided email.
    - Compares the stored password with the provided password.
    - **If Match:** Returns HTTP 200 (Success).
    - **If No Match:** Returns HTTP 401 (Unauthorized).
4. **Output:** On success, the frontend redirects the browser window to selection.html.

III. The Practice Loop (Core Feature)

This is the main functionality where the user practices specific sounds (e.g., 'S' words).

1. Selection Phase (selection.html)

- User chooses a specific lisp type (e.g., "S sounds", "R sounds").
- Clicking "S sounds" navigates to the practice page s.html.

2. Word Generation (s.html)

- **Trigger:** User clicks the "Next Word" button.
- **Action:** JavaScript selects a random string from the pre-defined wordList array (e.g., "sit", "soup", "seal").
- **Display:** The chosen word is rendered in the DOM (Document Object Model) for the user to read.

3. Audio Recording (s.html)

- **Trigger:** User clicks "Hold to Record".
- **Action:**
    - Browser requests microphone permission via navigator.mediaDevices.getUserMedia.
    - A MediaRecorder instance starts capturing audio chunks.
- **Stop:** User clicks the button again to stop recording. The chunks are assembled into a single audio Blob (type: audio/webm).

4. Processing & Transcription (app.py)

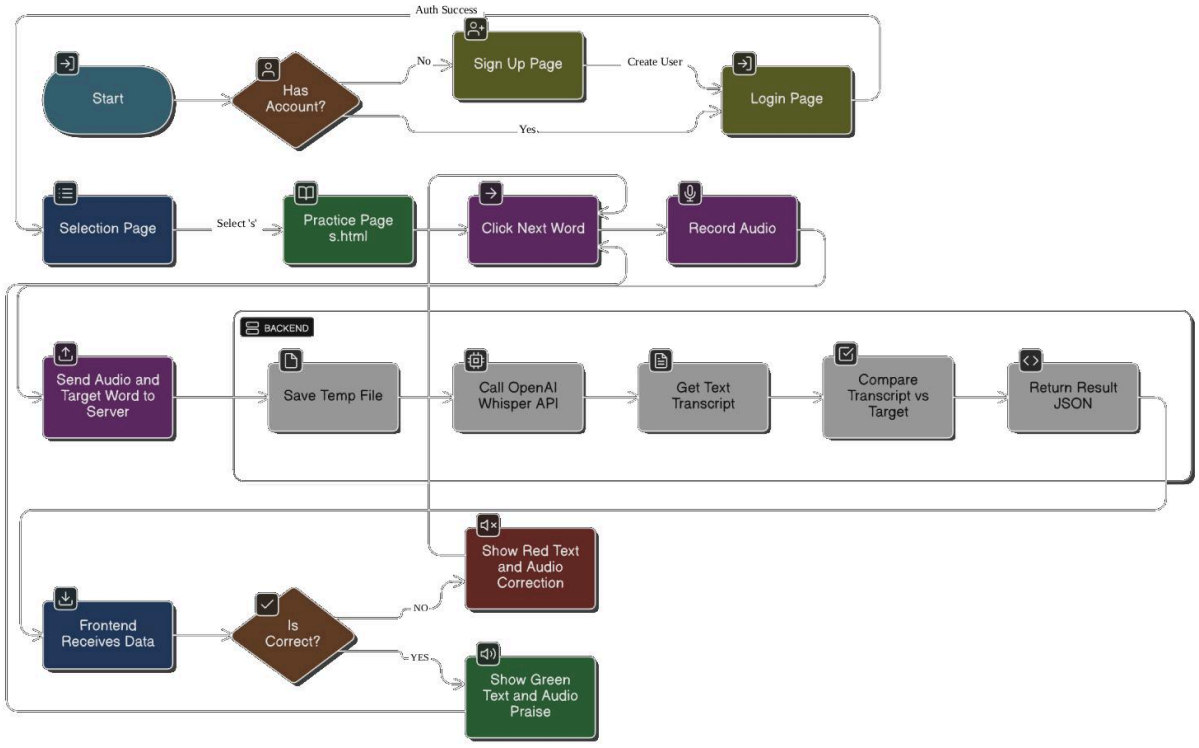
- **Transmission:** Frontend creates FormData containing the audio Blob and the target_word. It sends a POST request to /transcribe.
- **Server Logic:**
    1. **File Save:** The server temporarily saves the uploaded audio as temp_audio.webm.
    2. **AI Transcription:** The server reads the file and sends it to **OpenAI Whisper API** (model="whisper-1").
    3. **Response:** OpenAI returns the text string of what was spoken.
    4. **Accuracy Check:** The server calls check_accuracy(spoken_text, target_word):
        - Removes punctuation and converts both strings to lowercase.
        - Compares them for an exact match.
    5. **Cleanup:** The temporary audio file is deleted from the server.

6. **Return:** A JSON response is sent back containing: { success, transcription, is_correct }.

5. Feedback Loop (s.html)

- **Visual Feedback:**
  - If is_correct is **True**: Display the transcript in **Green**.
  - If is_correct is **False**: Display the transcript in **Red**.
- **Audio Feedback:** The browser uses the SpeechSynthesisUtterance API to speak to the user (e.g., "Great job! You said [word]" or "Try again. I heard [word]").

6.Flowchart



7.Program Code

index:

```
document.addEventListener('DOMContentLoaded', () => {
    // --- Configuration and Initialization ---


    // Target elements from the HTML
    const targetPhraseElement = document.getElementById('target-phrase');
    const startButton = document.getElementById('start-btn');
    const resultTextElement = document.getElementById('result-text');
    const feedbackMessageElement =
document.getElementById('feedback-message');


    let currentTargetPhrase = "";
    let isRecognizing = false;


    // Browser compatibility check and setup for Speech Recognition
    const SpeechRecognition = window.SpeechRecognition ||
window.webkitSpeechRecognition;


    // Check if the browser supports the required APIs
    if (!SpeechRecognition || !window.speechSynthesis) {
        startButton.disabled = true;
```

```
        feedbackMessageElement.textContent = "❌ Error: Your browser does not fully support the Web Speech API (Speech Recognition or Speech Synthesis). Please try Google Chrome or Microsoft Edge.";
        return;
    }

    // Initialize Speech Recognition
    const recognition = new SpeechRecognition();
    recognition.lang = 'en-US';        // Set the language
    recognition.continuous = false;    // Stop listening after a single, final result
    recognition.interimResults = false; // Only return the final, best guess
    recognition.maxAlternatives = 1;   // Only concerned with the top result

    // Initialize Speech Synthesis
    const synth = window.speechSynthesis;


    // --- Core Functions ---

    /**

    function speakPhrase(text) {
```

```
        // Clear any prior speech
        synth.cancel();

        const utterance = new SpeechSynthesisUtterance(text);

        // Optional: Set voice properties for a clearer, slower delivery suitable for therapy
        utterance.rate = 0.8; // Slightly slower pace
        utterance.pitch = 1.0;

        // Find a suitable voice (optional, but good for consistency)
        utterance.onend = () => {
            // Once the phrase is spoken, automatically start listening for the user's attempt
            startRecognition();
        };

        synth.speak(utterance);
    }

    /**
     * Cleans and normalizes strings for comparison.
```

```
 * Removes punctuation and converts to lower case.
 */
function normalizeText(text) {
    return text.toLowerCase().replace(/[.,/#!$%^&*;:{}=\-_`~()]/g, "").trim();
}

/**
 * Calculates the similarity (as a percentage) between two strings
 * using the Levenshtein distance (a simplified concept for this demo).
 * In a real app, you might use more complex phonetic matching or a paid API.
 * For this demo, we'll use a simple character-match score for illustration.
 */
function calculateSimilarity(target, spoken) {
    const normalizedTarget = normalizeText(target);
    const normalizedSpoken = normalizeText(spoken);

    if (normalizedTarget.length === 0 || normalizedSpoken.length === 0) {
        return 0;
    }

    // A very simple similarity score based on the length of the longest common substring
```

```
        let matchCount = 0;
        for (let i = 0; i < Math.min(normalizedTarget.length,
normalizedSpoken.length); i++) {
            if (normalizedTarget[i] === normalizedSpoken[i]) {
                matchCount++;
            }
        }

        // Calculate percentage based on the target length
        return Math.round((matchCount / normalizedTarget.length) * 100);
    }

    /**
     * Starts the listening service.
     */
    function startRecognition() {
        if (!isRecognizing) {
            resultTextElement.textContent = "Listening...";
            feedbackMessageElement.textContent = "Speak the phrase clearly now.";
            startButton.disabled = true;
            isRecognizing = true;
            recognition.start();
```

```
    }
}


// --- Event Handlers ---

// Handles the final transcription result
recognition.onresult = (event) => {
    const spokenTranscript = event.results[0][0].transcript;
    resultTextElement.textContent = `You Said: "${spokenTranscript}"`;

    const similarity = calculateSimilarity(currentTargetPhrase, spokenTranscript);

    let feedback;
    if (similarity >= 90) {
        feedback = `✅ Excellent! Similarity: ${similarity}%. Great job!`;
    } else if (similarity >= 60) {
        feedback = `👍 Good attempt! Similarity: ${similarity}%. Try to focus on the whole phrase.`;
    } else {
        feedback = `⚠️ Keep practicing. Similarity: ${similarity}%. Listen carefully and try again.`;
```

```
        }

        feedbackMessageElement.textContent = feedback;
    };

    // Event for when the speech service has ended (after speaking/error)
    recognition.onend = () => {
        isRecognizing = false;
        startButton.disabled = false;
        // The result handling takes care of the feedback
        // If the end event fires without a result (e.g., timeout), we show a message
        if (resultTextElement.textContent === "Listening...") {
            resultTextElement.textContent = "No speech detected. Press 'Start Practice' to try again.";
            feedbackMessageElement.textContent = "";
        }
    };

    // Handles errors (e.g., no microphone access, network issues)
    recognition.onerror = (event) => {
        isRecognizing = false;
        startButton.disabled = false;
```

```
        feedbackMessageElement.textContent = `❌ Recognition Error: ${event.error}. Please check your microphone and permissions.`;
        resultTextElement.textContent = "Error occurred during listening.";
    };


    // Handles the primary button click
    startButton.addEventListener('click', () => {
        // 1. Pick a new random phrase
        const randomIndex = Math.floor(Math.random() * practicePhrases.length);
        currentTargetPhrase = practicePhrases[randomIndex];

        // 2. Update the UI
        targetPhraseElement.textContent = currentTargetPhrase;
        resultTextElement.textContent = "Press 'Start Practice' to begin.";
        feedbackMessageElement.textContent = "Listen to the target phrase...";
        startButton.disabled = true; // Disable until the process is complete

        // 3. Speak the phrase (which will trigger recognition.start() on 'end')
        speakPhrase(currentTargetPhrase);
    });
```

```
    // Initial setup state
    targetPhraseElement.textContent = "Click 'Start Practice' to begin your session!";
    resultTextElement.textContent = "";
    feedbackMessageElement.textContent = "";
});
```

-app.py

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Mock database
users_db = []

@app.route("/")
def home():
    return "Speech Therapy App Backend Running"

@app.route("/signup", methods=["POST"])
def signup():
    data = request.get_json()

    username = data.get("username")
```

```
    email = data.get("email")
    password = data.get("password")

    if not username or not email or not password:
        return jsonify({"error": "Missing fields"}), 400

    for user in users_db:
        if user["email"] == email:
            return jsonify({"error": "User already exists"}), 409

    new_user = {
        "id": len(users_db) + 1,
        "username": username,
        "email": email,
        "password": password
    }

    users_db.append(new_user)
    return jsonify({"message": "User created successfully"}), 201

@app.route("/login", methods=["POST"])
def login():
    data = request.get_json()

    email = data.get("email")
```

```
    password = data.get("password")

    if not email or not password:
        return jsonify({"error": "Missing fields"}), 400

    for user in users_db:
        if user["email"] == email and user["password"] == password:
            return jsonify({"message": "Login successful"}), 200

    return jsonify({"error": "Invalid credentials"}), 401

if __name__ == "__main__":
    app.run(debug=True)
```

-index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Speech Therapy App</title>
</head>
<body>
    <h1>Welcome to Speech Therapy App</h1>
    <a href="login.html">Login</a><br>
    <a href="signup.html">Sign Up</a>
</body>
</html>
```

-login.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
</head>
<body>
    <h2>Login</h2>
    <form>
        Email: <input type="email"><br><br>
        Password: <input type="password"><br><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>
```

-singnup.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Sign Up</title>
</head>
<body>
   <h2>Sign Up</h2>
   <form>
      Full Name: <input type="text"><br><br>
      Email: <input type="email"><br><br>
      Username: <input type="text"><br><br>
      Password: <input type="password"><br><br>
      Confirm Password: <input type="password"><br><br>
      <button type="submit">Sign Up</button>
   </form>
</body>
</html>
```

-selection.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Selection</title>
</head>
<body>
    <h2>Select Sound to Practice</h2>
    <a href="s.html">S Sounds</a><br><br>
    <a href="r.html">R Sounds</a><br><br>
    <a href="t.html">T Sounds</a>
</body>
</html>
```

-s.html

```
<!DOCTYPE html>
<html>
<head>
    <title>S Sounds Practice</title>
</head>
<body>
    <h2>S Sound Practice</h2>
    <button onclick="getWord()">Get Random Word</button>
    <p id="word"></p>

    <script>
        const words = ["sun", "snake", "school", "star", "sand"];

        function getWord() {
            const randomIndex = Math.floor(Math.random() * words.length);
            document.getElementById("word").innerText = words[randomIndex];
        }
    </script>
</body>
</html>
```

-r.html

```
<!DOCTYPE html>
<html>
<head>
    <title>R Sounds Practice</title>
</head>
<body>
    <h2>R Sound Practice</h2>
    <button onclick="getWord()">Get Random Word</button>
    <p id="word"></p>

    <script>
        const words = ["rabbit", "road", "river", "rain", "ring"];

        function getWord() {
            const randomIndex = Math.floor(Math.random() * words.length);
            document.getElementById("word").innerText = words[randomIndex];
        }
    </script>
</body>
</html>
```

-t.html

```
<!DOCTYPE html>
<html>
<head>
   <title>T Sounds Practice</title>
</head>
<body>
   <h2>T Sound Practice</h2>
   <button onclick="getWord()">Get Random Word</button>
   <p id="word"></p>

   <script>
      const words = ["table", "tree", "time", "top", "toy"];

      function getWord() {
         const randomIndex = Math.floor(Math.random() * words.length);
         document.getElementById("word").innerText = words[randomIndex];
      }
   </script>
</body>
</html>
```

---

9. Input Output

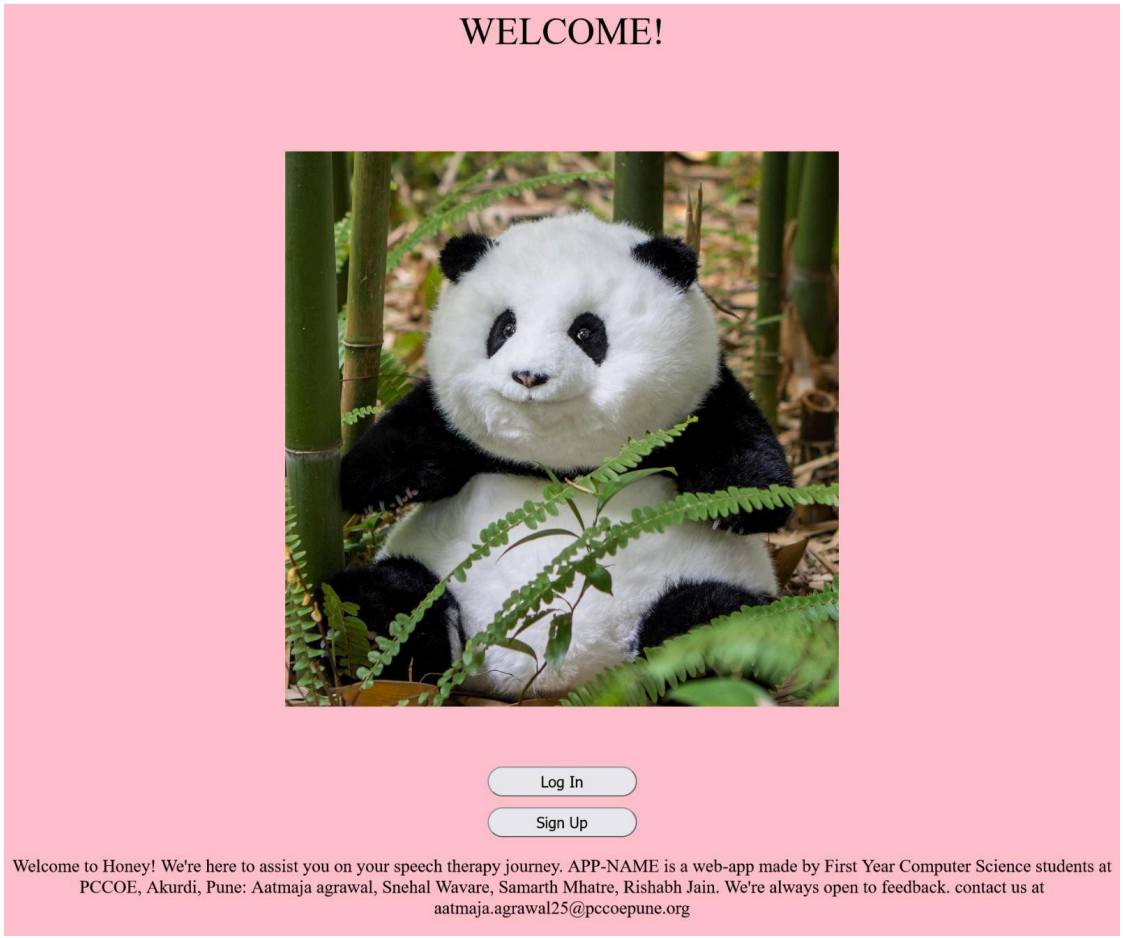


Fig1.Welcome page

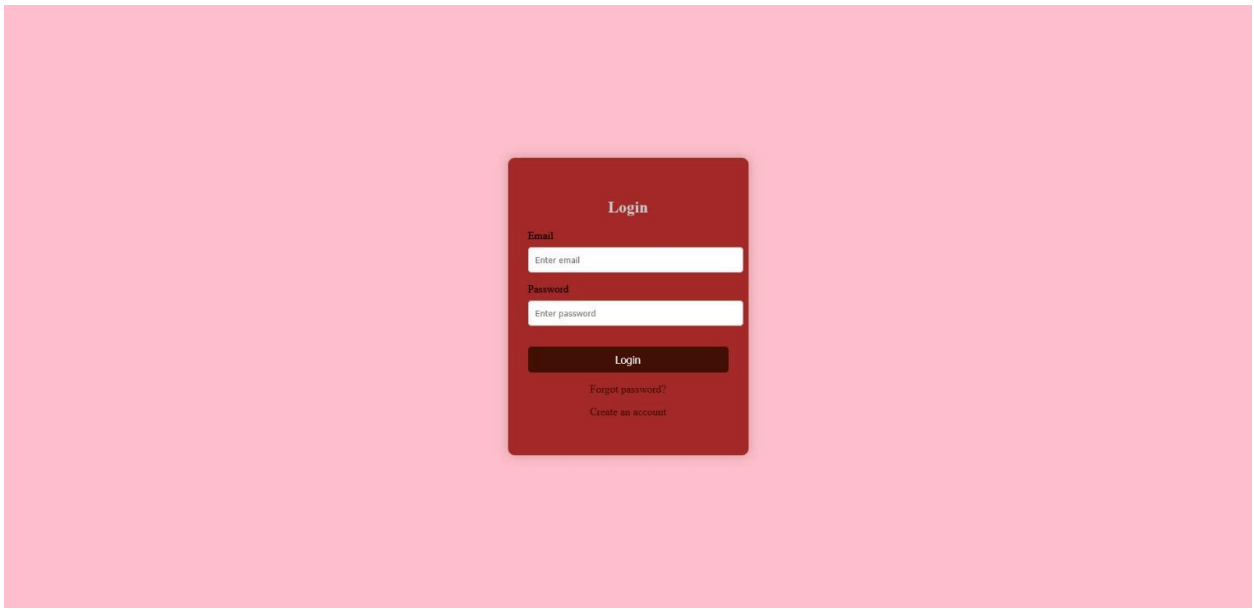


Fig2.Login page



Fig3.sign up page

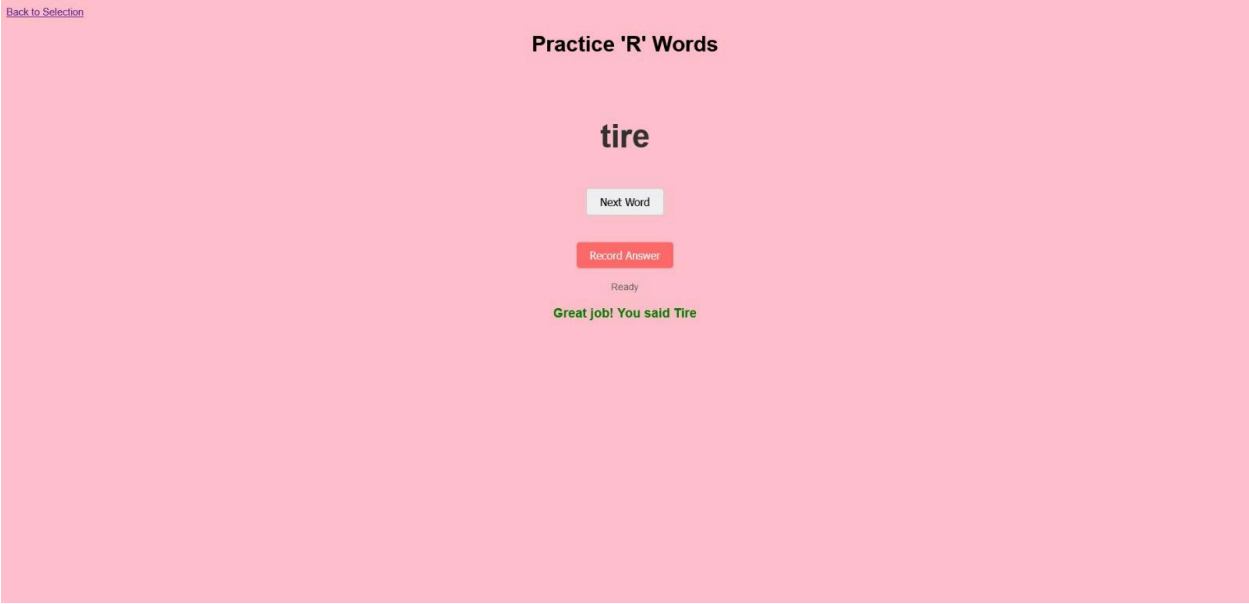


Fig4.Right pronunciation

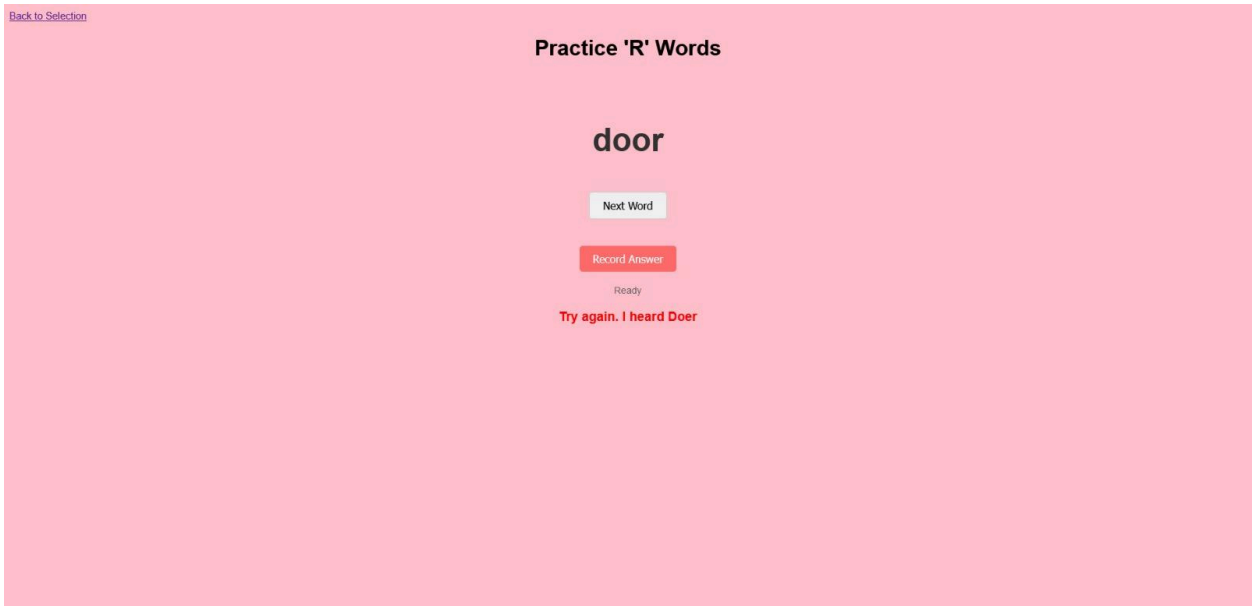


Fig5. Wrong pronunciation

10.Result and observation

- The program successfully created a web-based speech therapy application.
- The system was able to record the user's voice correctly.
- The speech input was processed properly by the program.
- The output given by the program was correct.
- The application provided accurate visual and audio feedback.
- The feedback helped users identify pronunciation mistakes.
- The program met the project objectives.
- Users could practice basic speech exercises easily.
- The application worked smoothly on supported web browsers.
- Overall, the program achieved its purpose and was useful for basic speech practice.

11. Learning Outcome

- Learned the basics of web development using HTML, CSS, and JavaScript
- Understood how to create and structure web pages using HTML
- Learned to design and style web pages using CSS
- Gained knowledge of adding interactivity to web pages using JavaScript
- Got hands-on experience in testing the application
- Learned to identify and fix errors in the code
- Improved coding skills and problem-solving ability through practical work

---

12. Conclusion

- This project developed a simple web-based speech therapy application.
- The app was made using HTML, CSS, JavaScript, and Python.
- It helps users practice pronunciation.
- The app provides visual and sound feedback to the users.
- This feedback helps in improving basic speech skills.
- The application worked properly and met the project objectives.
- It gave accurate feedback during practice.
- The app is easy to use for users.
- It runs smoothly on supported web browsers.
- In the future, the app can be improved further.
- User progress can be saved for better tracking.

- More speech exercises can be added.
- Improved speech recognition can be used.
- The app can be made mobile-friendly to support smartphones.

13.References

https://www.home-speech-home.com